

State Pattern

Category: Behavioural Design Pattern

1. The Problem

Consider a `DirectionService` class for a navigation app. It calculates ETA and provides directions between two points, and the logic differs based on the mode of transportation: Walking, Cycling, Car, or Train.

```
enum TransportationMode {
    WALKING, CYCLING, CAR, TRAIN
}

public class DirectionService {

    private TransportationMode mode;

    public DirectionService(TransportationMode mode) {
        this.mode = mode;
    }

    public void setMode(TransportationMode mode){
        this.mode = mode;
    }

    public int getETA(){
        switch(mode){
            case WALKING :
                return 10;
            case CYCLING :
                return 5;
            case CAR :
                return 2;
            case TRAIN :
                return 1;
            default:
                throw new IllegalArgumentException("Unknown Mode");
        }
    }

    public String getDirection(){
        switch (mode){
            case WALKING :
                return "Direction for walking: Head north for 500 meters";
            case CYCLING :
                return "Direction for cycling: Sidha chala chup chap";
            case CAR :
                return "Direction for Car: Use highway NH-66";
            case TRAIN :
                return "Direction for Train: Board the Red Line at Central Station";
            default:
                return "No directions available for the selected mode.";
        }
    }
}
```

}

Issues: - **Tight coupling & complex conditional logic** — `DirectionService` branches on a mode enum; as more modes are added, this grows harder to maintain. - **Violates the Open/Closed Principle** — adding a mode (e.g. Airplane) means modifying `DirectionService` directly. - **Code duplication** — similar blocks repeat per mode inside every method, increasing the chance of errors. - **Scalability issues** — the class keeps growing as modes/features are added, hurting readability.

2. The Solution — State Pattern

Intent: Allow an object to alter its behavior when its internal state changes, by delegating state-specific behavior to separate state objects — so the object appears to change its class at runtime.

- Each transportation mode becomes its own class implementing a common `TransportationMode` interface.
- `DirectionService` (the context) just holds a reference to the *current* mode/state object and delegates all mode-specific calls to it — no conditional logic required.

3. Key Idea

- **Replace conditionals with polymorphism** — instead of branching on an enum inside every method, each state/mode implements its own version of the behavior, and the context simply calls the interface method.
- **Program to an interface, not an implementation** — `DirectionService` only knows about the `TransportationMode` interface, never the concrete `WALKING`, `CAR`, etc. classes.
- The context's behavior *appears* to change at runtime purely because the object it delegates to has changed — no internal branching is needed at the call site.

4. Variants

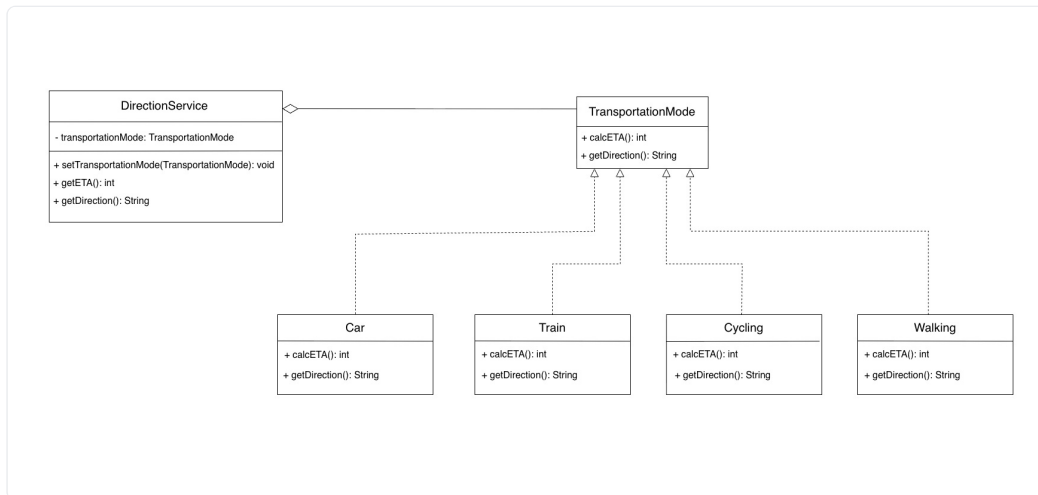
Not really applicable here — this is the standard State Pattern structure. In richer implementations, concrete states may also trigger their own transitions by holding a reference back to the context (not shown here, where the mode is set externally via `setMode()`).

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Context	Holds a reference to the current state and delegates behavior to it	<code>DirectionService</code>
State (interface)	Declares state-specific behavior all concrete states must implement	<code>TransportationMode</code>
Concrete State	Implements behavior for one particular state of the context	<code>WALKING</code> , <code>CYCLING</code> , <code>CAR</code> , <code>TRAIN</code>

Relationship type: Composition — `DirectionService` *holds* a `TransportationMode` (the diamond in the diagram points from `DirectionService` to `TransportationMode`).

Diagram



State Pattern UML diagram

`DirectionService` (context) composes a `TransportationMode` and exposes `setTransportationMode()`, `getETA()`, and `getDirection()`. `TransportationMode` is the state interface declaring `calcETA()` and `getDirection()`, implemented by the concrete states `Car`, `Train`, `Cycling`, and `Walking` — each providing its own mode-specific logic.

6. Code — Full Implementation

```
// State Interface
interface TransportationMode {
    public String getDirection();
    public String getETA();
}

class WALKING implements TransportationMode {
    @Override
    public String getDirection() {
        return "Direction for walking: Head north for 500 meters";
    }
    @Override
    public String getETA() {
        return "Calc ETA for WALKING is 10";
    }
}

class CYCLING implements TransportationMode {
    @Override
    public String getDirection() {
        return "Direction for cycling: Sidha chala chup chap";
    }
    @Override
    public String getETA() {
        return "Calc ETA for CYCLING is 5";
    }
}

// CAR and TRAIN follow the same shape as WALKING/CYCLING above,
// each returning its own direction text and ETA.
class CAR implements TransportationMode { /* ... */ }
class TRAIN implements TransportationMode { /* ... */ }
```

```

public class DirectionService {

    private TransportationMode transportationMode;

    public DirectionService(TransportationMode transportationMode) {
        this.transportationMode = transportationMode;
    }

    public void setMode(TransportationMode mode){
        this.transportationMode = mode;
    }

    // delegating the work to current state's concrete class
    public void getETA(){
        System.out.println(transportationMode.getETA());
    }

    public void getDirection(){
        System.out.println(transportationMode.getDirection());
    }
}

public class StatePatternExample {
    public static void main(String[] args) {
        DirectionService directionService = new DirectionService(new WALKING());
        directionService.getDirection();
        directionService.getETA();
    }
}

```

What changed vs. the “without pattern” version: - `TransportationMode` enum + `switch` statements in `getETA()` / `getDirection()` → `TransportationMode` interface implemented separately by `WALKING`, `CYCLING`, `CAR`, `TRAIN`. - All mode-specific logic lived inside `DirectionService` → each mode’s logic now lives in its own class; `DirectionService` only delegates. - Adding a new mode used to mean editing both `switch` statements in `DirectionService` → now it just means adding a new class implementing `TransportationMode`; `DirectionService` is untouched. - `mode` field changed from an enum value to an actual `TransportationMode` object, injected via constructor or `setMode()`.

7. Benefits

- **Eliminates conditional complexity** — no `switch` / `if-else` chains in the context class.
- **Open/Closed Principle** — new modes/states can be added as new classes without touching `DirectionService`.
- **Single Responsibility** — each state class owns exactly one mode’s behavior.
- **Cleaner state transitions** — switching state is just swapping which object the context holds (`setMode()`), rather than reassigning an enum and re-running conditional logic.

8. Drawbacks / Things to Watch Out For

- **More classes** — one per state/mode, which can add up for systems with many states.
- **Scattered logic** — behavior for one concept (e.g. transportation) is spread across multiple files instead of one place.
- Self-transitioning states may need a back-reference to the context, adding coupling.

9. Real-World Use Cases

- **UI Navigation** — an app's navigation behavior changes depending on state, e.g. `LoggedInState` vs. `LoggedOutState`.
- **UI Components** — buttons or controls that change behavior based on state (enabled, disabled, pressed).
- **Vending Machines** — states like waiting for money, dispensing product, or out of stock.
- **TCP Connections** — behavior changes based on connection state (listening, connected, closed).

10. Interview Questions & Answers

Q1. What is the State Pattern? A behavioural design pattern that lets an object alter its behavior when its internal state changes, by delegating state-specific behavior to separate state objects — making the object appear to change its class at runtime.

Q2. What problem does it solve? It removes complex, growing conditional logic (`switch / if-else`) tied to an internal state or mode, which otherwise leads to tight coupling, code duplication, and violations of the Open/Closed Principle whenever a new state is introduced.

Q3. What are its main components/participants? - **Context** — holds a reference to the current state and delegates behavior to it (`DirectionService`) - **State** — interface declaring state-specific operations (`TransportationMode`) - **Concrete State** — implements behavior for one specific state (`WALKING` , `CAR` , etc.)

Q4. How does the State Pattern differ from the Strategy Pattern, since their structures are nearly identical? Structurally identical (a context composing an interface implemented by interchangeable classes), but intent differs: **Strategy** is about choosing *how* to perform one operation, usually set once by the client. **State** represents an object's *internal condition*, which naturally changes over its lifetime — often the state itself triggers the transition to another state.

Q5. How are state transitions handled in the State Pattern? Simply by changing which concrete state object the context currently holds (e.g. `setMode()`). In more advanced implementations, the concrete state itself can trigger the transition by calling back into the context to swap itself out for another state.

Q6. What are the benefits? Eliminates conditional complexity, supports the Open/Closed Principle for adding new states, keeps each state's logic isolated (Single Responsibility), and makes state transitions explicit and clean.

Q7. What are the drawbacks? More classes to maintain (one per state), logic for a single concept becomes spread across multiple files, and concrete states may need a reference back to the context if they trigger their own transitions.

Q8. Can you give real-world examples? Login/logout navigation state in an app, UI components with enabled/disabled/pressed states, vending machine states, and TCP connection states (listening, connected, closed).